

```
"use client";

import React, { createContext, useContext, useState, useEffect } from
"react";
import { Check, AlertCircle, X, Info } from "lucide-react";
import { createClient } from "@lib/supabase/client";
import { toast } from "sonner";

// --- TYPES FROM SCHEMA (erd.md) ---
export type VerificationStatus =
  | "PENDING_ADMIN_REVIEW"
  | "APPROVED"
  | "REJECTED"
  | "NEEDS_CLARIFICATION"
  | "NEEDS_RESUBMISSION";

export type DocumentVerificationStatus = "PENDING" | "APPROVED" |
"REJECTED";

export interface Address {
  street: string;
  barangay: string;
  municipality: string;
  province: string;
  region: string;
  zip_code: string;
  latitude: number;
  longitude: number;
}

export interface IdDocument {
  id: string;
  id_type: string;
  file_url: string;
  verification_status: DocumentVerificationStatus;
  rejection_reason?: string;
  upload_date: string;
}

export interface SeniorProfile {
```

```

    id: string;
    phone: string;
    full_name: string;
    birthdate: string;
    age: number;
    verification_status: VerificationStatus;
    language_preference: "en" | "tl";
    sms_notifications: boolean;
    created_at: string;
    id_document: IdDocument;
    id_document_back?: IdDocument;
    address: Address;
    internal_notes: string;
    resubmit_fields?: string[];
    medicine_requests_count?: number;
    assistance_requests_count?: number;
    pending_resubmission?: {
      id: string;
      resubmitted_data: {
        full_name?: string;
        street?: string;
        barangay?: string;
        latitude?: number;
        longitude?: number;
        id_front_url?: string;
        id_back_url?: string;
        pin?: string;
      };
      created_at: string;
    };
  }
}

```

```

export interface Medicine {
  id: string;
  name: string;
  generic_name: string;
  description: string;
  dosage_strength: string;
  unit: string;
  usage_instructions: string;
}

```

```
    available_quantity: number;
    is_active: boolean;
}

export type MedicineRequestStatus =
  | "PENDING"
  | "APPROVED"
  | "COMPLETED"
  | "REJECTED";

export interface MedicineRequest {
  id: string;
  user_id: string;
  medicine_id: string;
  quantity: number;
  reason: string;
  prescription_url?: string;
  status: MedicineRequestStatus;
  pharmacist_notes?: string;
  request_date: string;
  dispense_date?: string;
}

export type AssistanceRequestStatus =
  | "PENDING"
  | "IN_PROGRESS"
  | "COMPLETED"
  | "REJECTED";

export type AssistanceCategory =
  | "Medicine"
  | "Healthcare"
  | "Food"
  | "Transport"
  | "Social"
  | "Other";

export interface AssistanceRequest {
  id: string;
  user_id: string;
```

```
    category: AssistanceCategory;
    description: string;
    urgency_level: "Normal" | "Urgent";
    status: AssistanceRequestStatus;
    assigned_to?: string | null;
    created_at: string;
}

export type DigitalIdStatus = "ACTIVE" | "EXPIRED" | "SUSPENDED";

export interface DigitalId {
    id: string;
    user_id: string;
    id_number: string;
    qr_code_url: string;
    issue_date: string;
    expiry_date: string;
    status: DigitalIdStatus;
}

export interface AdminLog {
    id: string;
    admin_id: string;
    admin_name: string;
    action: string;
    target_id: string;
    target_type: string;
    details: Record<string, any>;
    status: "SUCCESS" | "ERROR";
    created_at: string;
}

export interface ToastMessage {
    id: string;
    message: string;
    type: "success" | "error" | "info" | "warning";
}

interface AdminContextType {
    seniors: SeniorProfile[];
}
```

```

medicines: Medicine[];
medicineRequests: MedicineRequest[];
assistanceRequests: AssistanceRequest[];
digitalIds: DigitalId[];
logs: AdminLog[];
toasts: ToastMessage[];
addToast: (message: string, type?: ToastMessage["type"]) => void;
removeToast: (id: string) => void;

// Archived items
archivedSeniors: SeniorProfile[];
archivedMedicines: Medicine[];

// Actions
verifySenior: (id: string) => void;
rejectSenior: (id: string, reason: string) => void;
flagSenior: (id: string, notes: string) => void;
updateSeniorNotes: (id: string, notes: string) => void;
deactivateSenior: (id: string) => void;
restoreSenior: (id: string) => void;
deleteSeniorPermanently: (id: string) => void;
generateDigitalId: (userId: string) => void;
batchGenerateDigitalIds: () => void;
suspendDigitalId: (id: string) => void;
renewDigitalId: (id: string) => void;
requestResubmission: (id: string, fields: string[], reason: string) =>
Promise<void>;
approveResubmission: (id: string) => Promise<void>;
addMedicine: (medicine: Omit<Medicine, "id">) => void;
updateMedicine: (id: string, medicine: Partial<Medicine>) => void;
deleteMedicine: (id: string) => void;
restoreMedicine: (id: string) => void;
deleteMedicinePermanently: (id: string) => void;
updateMedicineRequestStatus: (
  id: string,
  status: MedicineRequestStatus,
  pharmacistNotes?: string
) => void;
updateAssistanceRequestStatus: (
  id: string,

```

```

        status: AssistanceRequestStatus,
        notes?: string,
        assignedTo?: string | null
    ) => void;
}

const AdminContext = createContext<AdminContextType |
undefined>(undefined);

// --- INITIAL MOCK DATA REMOVED (Real database query is used) ---

export function AdminProvider({ children }: { children: React.ReactNode })
{
    const [seniors, setSeniors] = useState<SeniorProfile[]>([]);
    const [medicines, setMedicines] = useState<Medicine[]>([]);
    const [medicineRequests, setMedicineRequests] =
useState<MedicineRequest[]>([]);
    const [assistanceRequests, setAssistanceRequests] =
useState<AssistanceRequest[]>([]);
    const [digitalIds, setDigitalIds] = useState<DigitalId[]>([]);
    const [logs, setLogs] = useState<AdminLog[]>([]);
    const [toasts, setToasts] = useState<ToastMessage[]>([]);
    const [archivedSeniors, setArchivedSeniors] =
useState<SeniorProfile[]>([]);
    const [archivedMedicines, setArchivedMedicines] =
useState<Medicine[]>([]);

    // Load from Supabase on mount
    useEffect(() => {
        async function loadData() {
            const supabase = createClient();

            try {
                const [
                    { data: profiles },
                    { data: meds },
                    { data: medReqs },
                    { data: astReqs },
                    { data: digIds },
                    { data: adminLogs },

```

```

    { data: resubs },
  ] = await Promise.all([
    supabase.from("user_profiles").select("*", user_addresses(*),
id_documents(*)),
    supabase.from("medicines").select("*"),
    supabase.from("medicine_requests").select("*"),
    supabase.from("assistance_requests").select("*"),
    supabase.from("digital_ids").select("*"),
    supabase.from("admin_logs").select("*").order("created_at", {
ascending: false }),
    supabase.from("resubmissions").select("*").eq("status",
"PENDING"),
  ]);

  if (meds) {
    setMedicines(meds);
  }

  if (profiles) {
    const active: SeniorProfile[] = [];
    const archived: SeniorProfile[] = [];

    profiles.forEach((p: any) => {
      const addr = p.user_addresses?.[0] || {
        street: "",
        barangay: "",
        municipality: "San Luis",
        province: "Pampanga",
        region: "Region III",
        zip_code: "2014",
        latitude: 15.0253,
        longitude: 120.7854,
      };

      const frontDoc = p.id_documents?.find((d: any) => d.id_type
=== "SENIOR_CITIZEN_ID_FRONT") || p.id_documents?.[0] || {
        id: "",
        id_type: "SENIOR_CITIZEN_ID_FRONT",
        file_url: "",
        verification_status: "PENDING",
        upload_date: new Date().toISOString(),

```

```

    };

    const backDoc = p.id_documents?.find((d: any) => d.id_type ===
"SENIOR_CITIZEN_ID_BACK");
    const userMedReqs = medReqs?.filter((r: any) => r.user_id ===
p.id) || [];
    const userAstReqs = astReqs?.filter((r: any) => r.user_id ===
p.id) || [];

    const mapped: SeniorProfile = {
      id: p.id,
      phone: p.phone,
      full_name: p.full_name || "New Citizen",
      birthdate: p.birthdate || "",
      age: p.age || 0,
      verification_status: p.verification_status,
      language_preference: p.language_preference === "Tagalog" ||
p.language_preference === "tl" ? "tl" : "en",
      sms_notifications: p.sms_notifications ?? true,
      created_at: p.created_at,
      id_document: {
        id: frontDoc.id,
        id_type: frontDoc.id_type || "",
        file_url: frontDoc.file_url || "",
        verification_status: frontDoc.verification_status as
DocumentVerificationStatus,
        rejection_reason: frontDoc.rejection_reason || undefined,
        upload_date: frontDoc.upload_date,
      },
      id_document_back: backDoc ? {
        id: backDoc.id,
        id_type: backDoc.id_type || "",
        file_url: backDoc.file_url || "",
        verification_status: backDoc.verification_status as
DocumentVerificationStatus,
        rejection_reason: backDoc.rejection_reason || undefined,
        upload_date: backDoc.upload_date,
      } : undefined,
      address: {
        street: addr.street || "",
        barangay: addr.barangay || "",

```

```

        municipality: addr.municipality || "",
        province: addr.province || "",
        region: addr.region || "",
        zip_code: addr.zip_code || "",
        latitude: addr.latitude || 0,
        longitude: addr.longitude || 0,
    },
    medicine_requests_count: userMedReqs.length,
    assistance_requests_count: userAstReqs.length,
    internal_notes: p.internal_notes || "",
    resubmit_fields: p.resubmit_fields || [],
    pending_resubmission: (() => {
        const resub = resubs?.find((r: any) => r.user_id ===
p.id);

        return resub ? {
            id: resub.id,
            resubmitted_data: resub.resubmitted_data,
            created_at: resub.created_at,
        } : undefined;
    })(),
    };

    if (p.verification_status === "ARCHIVED") {
        archived.push(mapped);
    } else {
        active.push(mapped);
    }
});

setSeniors(active);
setArchivedSeniors(archived);
}

if (meds) {
    const activeMeds = meds.filter((m: any) => m.is_active ===
true);

    const archivedMeds = meds.filter((m: any) => m.is_active ===
false);

    setMedicines(activeMeds);
    setArchivedMedicines(archivedMeds);
}

```

```

    }

    if (medReqs) {
      setMedicineRequests(medReqs);
    }

    if (astReqs) {
      setAssistanceRequests(astReqs);
    }

    if (digIds) {
      setDigitalIds(digIds);
    }

    if (adminLogs) {
      setLogs(
        adminLogs.map((l: any) => ({
          id: l.id,
          admin_id: l.admin_id,
          admin_name: "MSWD Admin",
          action: l.action,
          target_id: l.target_id || "",
          target_type: l.target_type || "",
          details: l.details || {},
          status: l.status || "SUCCESS",
          created_at: l.created_at,
        })))
      );
    }
  } catch (err) {
    console.error("Failed to fetch data from Supabase:", err);
  }
}

loadData();
}, []);

// Save to localStorage whenever state changes
const saveToLocal = (key: string, data: any) => {
  localStorage.setItem(key, JSON.stringify(data));
};

const addToast = (message: string, type: ToastMessage["type"] =
"success") => {

```

```

    if (type === "success") {
      toast.success(message);
    } else if (type === "error") {
      toast.error(message);
    } else if (type === "warning") {
      toast.warning(message);
    } else {
      toast.info(message);
    }
  };

const removeToast = (id: string) => {
  // No-op since we migrated to Sonner
};

const addLog = (
  action: string,
  targetId: string,
  targetType: string,
  details: Record<string, any>,
  status: "SUCCESS" | "ERROR" = "SUCCESS"
) => {
  const newLog: AdminLog = {
    id: "log-" + Math.random().toString(36).substring(2, 9),
    admin_id: "AD-01",
    admin_name: "MSWD Admin AD",
    action,
    target_id: targetId,
    target_type: targetType,
    details,
    status,
    created_at: new Date().toISOString(),
  };
  setLogs((prev) => {
    const updated = [newLog, ...prev];
    saveToLocal("mswd_logs", updated);
    return updated;
  });
};

```

```

// VERIFY SENIOR
const verifySenior = async (id: string) => {
  const updatedSeniors = seniors.map((s) => {
    if (s.id === id) {
      return {
        ...s,
        verification_status: "APPROVED" as VerificationStatus,
        id_document: {
          ...s.id_document,
          verification_status: "APPROVED" as DocumentVerificationStatus,
        },
      };
    }
    return s;
  });

  setSeniors(updatedSeniors);
  saveToLocal("mswd_seniors", updatedSeniors);

  const senior = seniors.find((s) => s.id === id);
  addToast(`Approved ${senior?.full_name ?? "Senior"}. SMS sent successfully.`, "success");

  addLog(
    "USER_VERIFIED",
    id,
    "user_profiles",
    { name: senior?.full_name, phone: senior?.phone }
  );

  try {
    const supabase = createClient();
    await supabase.from("user_profiles").update({ verification_status: "APPROVED" }).eq("id", id);
    await supabase.from("id_documents").update({ verification_status: "APPROVED" }).eq("user_id", id);

    // Automatically generate Digital ID upon verification approval
    await generateDigitalId(id);
  } catch (e) {

```

```

    console.error("DB update error verifySenior:", e);
  }

  // Auto-generate Digital ID after approval
  generateDigitalId(id);
};

// REJECT SENIOR
const rejectSenior = async (id: string, reason: string) => {
  const updatedSeniors = seniors.map((s) => {
    if (s.id === id) {
      return {
        ...s,
        verification_status: "REJECTED" as VerificationStatus,
        id_document: {
          ...s.id_document,
          verification_status: "REJECTED" as DocumentVerificationStatus,
          rejection_reason: reason,
        },
      };
    }
  });
  return s;
});

setSeniors(updatedSeniors);
saveToLocal("mswd_seniors", updatedSeniors);

const senior = seniors.find((s) => s.id === id);
addToast(`Rejected onboarding for ${senior?.full_name ?? "Senior"}.
Reason sent via SMS.`, "warning");

addLog(
  "USER_REJECTED",
  id,
  "user_profiles",
  { name: senior?.full_name, phone: senior?.phone, reason }
);

try {
  const supabase = createClient();

```

```

        await supabase.from("user_profiles").update({ verification_status:
"REJECTED" }).eq("id", id);
        await supabase.from("id_documents").update({ verification_status:
"REJECTED", rejection_reason: reason }).eq("user_id", id);
    } catch (e) {
        console.error("DB update error rejectSenior:", e);
    }
};

// FLAG / UPDATE INTERNAL NOTES
const flagSenior = async (id: string, notes: string) => {
    const updatedSeniors = seniors.map((s) => {
        if (s.id === id) {
            return {
                ...s,
                verification_status: "NEEDS_CLARIFICATION" as
VerificationStatus,
                internal_notes: notes,
            };
        }
        return s;
    });
    setSeniors(updatedSeniors);
    saveToLocal("mswd_seniors", updatedSeniors);

    const senior = seniors.find((s) => s.id === id);
    addToast(`Flagged ${senior?.full_name ?? "Senior"} for
clarification.`, "info");

    addLog(
        "USER_UPDATED",
        id,
        "user_profiles",
        { name: senior?.full_name, flag_status: "NEEDS_CLARIFICATION", notes
}
    );
};

try {
    const supabase = createClient();

```

```

    await supabase.from("user_profiles").update({ verification_status:
"NEEDS_CLARIFICATION", internal_notes: notes }).eq("id", id);
  } catch (e) {
    console.error("DB update error flagSenior:", e);
  }
};

// UPDATE INTERNAL NOTES ONLY
const updateSeniorNotes = async (id: string, notes: string) => {
  const updatedSeniors = seniors.map((s) => {
    if (s.id === id) {
      return {
        ...s,
        internal_notes: notes,
      };
    }
    return s;
  });
  setSeniors(updatedSeniors);
  saveToLocal("mswd_seniors", updatedSeniors);

  const senior = seniors.find((s) => s.id === id);
  addLog(
    "USER_UPDATED",
    id,
    "user_profiles",
    { name: senior?.full_name, notes }
  );

  try {
    const supabase = createClient();
    await supabase.from("user_profiles").update({ internal_notes: notes
}).eq("id", id);
  } catch (e) {
    console.error("DB update error updateSeniorNotes:", e);
  }
};

// ARCHIVE/DEACTIVATE SENIOR
const deactivateSenior = async (id: string) => {

```

```

const senior = seniors.find((s) => s.id === id);
if (!senior) return;

const updatedSeniors = seniors.filter((s) => s.id !== id);
setSeniors(updatedSeniors);
saveToLocal("mswd_seniors", updatedSeniors);

setArchivedSeniors((prev) => {
  const updated = [...prev, senior];
  saveToLocal("mswd_archived_seniors", updated);
  return updated;
});

addToast(`Deactivated and archived profile for ${senior.full_name}.`,
"error");

addLog(
  "USER_DEACTIVATED",
  id,
  "user_profiles",
  { name: senior.full_name }
);

try {
  const supabase = createClient();
  await supabase.from("user_profiles").update({ verification_status:
"ARCHIVED" }).eq("id", id);
} catch (e) {
  console.error("DB update error deactivateSenior:", e);
}
};

// RESTORE SENIOR
const restoreSenior = async (id: string) => {
  const senior = archivedSeniors.find((s) => s.id === id);
  if (!senior) return;

  const updatedArchived = archivedSeniors.filter((s) => s.id !== id);
  setArchivedSeniors(updatedArchived);
  saveToLocal("mswd_archived_seniors", updatedArchived);

```

```

    setSeniors((prev) => {
      const updated = [...prev, { ...senior, verification_status:
"APPROVED" as VerificationStatus }]];
      saveToLocal("mswd_seniors", updated);
      return updated;
    });

    addToast(`Restored profile for ${senior.full_name} to active list.`,
"success");

    addLog(
      "USER_RESTORED",
      id,
      "user_profiles",
      { name: senior.full_name }
    );

    try {
      const supabase = createClient();
      await supabase.from("user_profiles").update({ verification_status:
"APPROVED" }).eq("id", id);
    } catch (e) {
      console.error("DB update error restoreSenior:", e);
    }
  };

  // DELETE SENIOR PERMANENTLY
  const deleteSeniorPermanently = async (id: string) => {
    const senior = archivedSeniors.find((s) => s.id === id);
    if (!senior) return;

    const updatedArchived = archivedSeniors.filter((s) => s.id !== id);
    setArchivedSeniors(updatedArchived);
    saveToLocal("mswd_archived_seniors", updatedArchived);

    addToast(`Permanently deleted profile for ${senior.full_name}.`,
"error");

    addLog(

```

```

    "USER_DELETED_PERMANENTLY",
    id,
    "user_profiles",
    { name: senior.full_name }
  );

  try {
    const supabase = createClient();
    await supabase.from("user_profiles").delete().eq("id", id);
  } catch (e) {
    console.error("DB delete error deleteSeniorPermanently:", e);
  }
};

// GENERATE DIGITAL ID
const generateDigitalId = async (userId: string) => {
  // Check if ID already exists
  if (digitalIds.some((d) => d.user_id === userId && d.status ===
"ACTIVE")) {
    return;
  }

  const existingIdNumbers = digitalIds.map((d) => d.id_number);
  const generateUniqueIdNumber = (existing: string[]): string => {
    let attempts = 0;
    while (attempts < 1000) {
      const num = Math.floor(1000000000 + Math.random() *
9000000000).toString();
      if (!existing.includes(num)) return num;
      attempts++;
    }
    return Math.floor(1000000000 + Math.random() * 9000000000).toString();
  };

  const idNumber = generateUniqueIdNumber(existingIdNumbers);
  const issueDate = new Date().toISOString().split("T")[0];
  const expiryDate = new Date(Date.now() + 5 * 365 *
86400000).toISOString().split("T")[0]; // +5 Years

  const newId: DigitalId = {

```

```

    id: "id-" + Math.random().toString(36).substring(2, 9),
    user_id: userId,
    id_number: idNumber,
    qr_code_url: idNumber,
    issue_date: issueDate,
    expiry_date: expiryDate,
    status: "ACTIVE",
  };

  setDigitalIds((prev) => {
    const updated = [...prev, newId];
    saveToLocal("mswd_digital_ids", updated);
    return updated;
  });

  const senior = seniors.find((s) => s.id === userId);
  addToast(`Generated Digital ID ${idNumber} for ${senior?.full_name ??
"Senior"}.`, "success");

  addLog(
    "DIGITAL_ID_GENERATED",
    newId.id,
    "digital_ids",
    { user_id: userId, id_number: idNumber, name: senior?.full_name }
  );

  try {
    const supabase = createClient();
    await supabase.from("digital_ids").insert({
      user_id: userId,
      id_number: idNumber,
      qr_code_url: idNumber,
      issue_date: issueDate,
      expiry_date: expiryDate,
      status: "ACTIVE",
    });
  } catch (e) {
    console.error("DB update error generateDigitalId:", e);
  }
};

```

```

// BATCH GENERATE FOR ALL VERIFIED SENIORS WITHOUT ACTIVE IDS
const batchGenerateDigitalIds = async () => {
  const verifiedSeniorsWithoutId = seniors.filter(
    (s) =>
      s.verification_status === "APPROVED" &&
      !digitalIds.some((d) => d.user_id === s.id && d.status ===
"ACTIVE")
  );

  if (verifiedSeniorsWithoutId.length === 0) {
    addToast("No new verified seniors without active IDs found.",
"info");
    return;
  }

  const existingIdNumbers = [...digitalIds.map((d) => d.id_number)];
  const generateUniqueIdNumber = (existing: string[]): string => {
    let attempts = 0;
    while (attempts < 1000) {
      const num = Math.floor(1000000000 + Math.random() *
9000000000).toString();
      if (!existing.includes(num)) return num;
      attempts++;
    }
    return Math.floor(1000000000 + Math.random() * 9000000000).toString();
  };

  const newIds: DigitalId[] = [];
  const issueDate = new Date().toISOString().split("T")[0];
  const expiryDate = new Date(Date.now() + 5 * 365 *
86400000).toISOString().split("T")[0];

  verifiedSeniorsWithoutId.forEach((s) => {
    const idNumber = generateUniqueIdNumber(existingIdNumbers);
    existingIdNumbers.push(idNumber);

    newIds.push({
      id: "id-" + Math.random().toString(36).substring(2, 9),
      user_id: s.id,

```

```

        id_number: idNumber,
        qr_code_url: idNumber,
        issue_date: issueDate,
        expiry_date: expiryDate,
        status: "ACTIVE",
    });

    addLog(
        "DIGITAL_ID_GENERATED",
        s.id,
        "digital_ids",
        { user_id: s.id, id_number: idNumber, name: s.full_name }
    );
});

setDigitalIds((prev) => {
    const updated = [...prev, ...newIds];
    saveToLocal("mswd_digital_ids", updated);
    return updated;
});

addToast(`Batch generated ${newIds.length} Digital IDs successfully.`,
"success");

try {
    const supabase = createClient();
    await supabase.from("digital_ids").insert(
        newIds.map((item) => ({
            user_id: item.user_id,
            id_number: item.id_number,
            qr_code_url: item.qr_code_url,
            issue_date: item.issue_date,
            expiry_date: item.expiry_date,
            status: "ACTIVE",
        })))
    );
} catch (e) {
    console.error("DB update error batchGenerateDigitalIds:", e);
}
};

```

```

// SUSPEND DIGITAL ID
const suspendDigitalId = async (id: string) => {
  const digId = digitalIds.find((d) => d.id === id);
  if (!digId) return;

  const updatedIds = digitalIds.map((d) => {
    if (d.id === id) {
      return { ...d, status: "SUSPENDED" as const };
    }
    return d;
  });
  setDigitalIds(updatedIds);
  saveToLocal("mswd_digital_ids", updatedIds);

  const senior = seniors.find((s) => s.id === digId.user_id);
  addToast(`Suspended Digital ID ${digId.id_number} for
${senior?.full_name ?? "Senior"}`, "error");

  addLog(
    "DIGITAL_ID_SUSPENDED",
    id,
    "digital_ids",
    { id_number: digId.id_number, name: senior?.full_name }
  );

  try {
    const supabase = createClient();
    await supabase.from("digital_ids").update({ status: "SUSPENDED"
}).eq("id", id);
  } catch (e) {
    // Try mapping to user_id match if id is client-only uuid
    const supabase = createClient();
    await supabase.from("digital_ids").update({ status: "SUSPENDED"
}).eq("user_id", digId.user_id);
  }
};

// RENEW DIGITAL ID
const renewDigitalId = async (id: string) => {

```

```

const digId = digitalIds.find((d) => d.id === id);
if (!digId) return;

const issueDate = new Date().toISOString().split("T")[0];
const expiryDate = new Date(Date.now() + 5 * 365 *
86400000).toISOString().split("T")[0];

const updatedIds = digitalIds.map((d) => {
  if (d.id === id) {
    return {
      ...d,
      issue_date: issueDate,
      expiry_date: expiryDate,
      status: "ACTIVE" as const,
    };
  }
  return d;
});
setDigitalIds(updatedIds);
saveToLocal("mswd_digital_ids", updatedIds);

const senior = seniors.find((s) => s.id === digId.user_id);
addToast(`Renewed Digital ID ${digId.id_number} for
${senior?.full_name ?? "Senior"}`, "success");

addLog(
  "DIGITAL_ID_RENEWED",
  id,
  "digital_ids",
  { id_number: digId.id_number, name: senior?.full_name }
);

try {
  const supabase = createClient();
  await supabase
    .from("digital_ids")
    .update({
      issue_date: issueDate,
      expiry_date: expiryDate,
      status: "ACTIVE",
    });
}

```

```

    })
    .eq("id", id);
} catch (e) {
  const supabase = createClient();
  await supabase
    .from("digital_ids")
    .update({
      issue_date: issueDate,
      expiry_date: expiryDate,
      status: "ACTIVE",
    })
    .eq("user_id", digId.user_id);
}
};

// ADD MEDICINE
const addMedicine = async (med: Omit<Medicine, "id">) => {
  const tempId = "med-" + Math.random().toString(36).substring(2, 9);
  const newMed: Medicine = {
    ...med,
    id: tempId,
  };
  setMedicines((prev) => {
    const updated = [...prev, newMed];
    saveToLocal("mswd_medicines", updated);
    return updated;
  });
  addToast(`Added medicine ${newMed.name} to inventory.`, "success");

  addLog("MEDICINE_ADDED", tempId, "medicines", {
    name: newMed.name,
    quantity: newMed.available_quantity,
  });

  try {
    const supabase = createClient();
    await supabase.from("medicines").insert({
      name: med.name,
      generic_name: med.generic_name,
      description: med.description,
    });
  } catch (e) {
    console.log(e);
  }
};

```

```

        dosage_strength: med.dosage_strength,
        unit: med.unit,
        usage_instructions: med.usage_instructions,
        available_quantity: med.available_quantity,
        is_active: med.is_active,
    });
} catch (e) {
    console.error("DB update error addMedicine:", e);
}
};

// UPDATE MEDICINE
const updateMedicine = async (id: string, fields: Partial<Medicine>) =>
{
    const updatedMeds = medicines.map((m) => {
        if (m.id === id) {
            return { ...m, ...fields };
        }
        return m;
    });
    setMedicines(updatedMeds);
    saveToLocal("mswd_medicines", updatedMeds);

    const med = medicines.find((m) => m.id === id);
    addToast(`Updated inventory details for ${med?.name}.`, "success");

    addLog("MEDICINE_UPDATED", id, "medicines", {
        name: med?.name,
        fields_changed: Object.keys(fields),
    });

    try {
        const supabase = createClient();
        await supabase.from("medicines").update(fields).eq("id", id);
    } catch (e) {
        console.error("DB update error updateMedicine:", e);
    }
};

// DELETE MEDICINE

```

```

const deleteMedicine = async (id: string) => {
  const med = medicines.find((m) => m.id === id);
  if (!med) return;

  const updatedMeds = medicines.filter((m) => m.id !== id);
  setMedicines(updatedMeds);
  saveToLocal("mswd_medicines", updatedMeds);

  setArchivedMedicines((prev) => {
    const updated = [...prev, med];
    saveToLocal("mswd_archived_medicines", updated);
    return updated;
  });

  addToast(`Archived medicine ${med.name} from inventory.`, "warning");

  addLog("MEDICINE_ARCHIVED", id, "medicines", { name: med.name });

  try {
    const supabase = createClient();
    await supabase.from("medicines").update({ is_active: false
  }).eq("id", id);
  } catch (e) {
    console.error("DB update error deleteMedicine:", e);
  }
};

// RESTORE MEDICINE
const restoreMedicine = async (id: string) => {
  const med = archivedMedicines.find((m) => m.id === id);
  if (!med) return;

  const updatedArchived = archivedMedicines.filter((m) => m.id !== id);
  setArchivedMedicines(updatedArchived);
  saveToLocal("mswd_archived_medicines", updatedArchived);

  setMedicines((prev) => {
    const updated = [...prev, med];
    saveToLocal("mswd_medicines", updated);
    return updated;
  });
};

```

```

    });

    addToast(`Restored medicine ${med.name} to active inventory.`,
"success");

    addLog("MEDICINE_RESTORED", id, "medicines", { name: med.name });

    try {
        const supabase = createClient();
        await supabase.from("medicines").update({ is_active: true
}).eq("id", id);
    } catch (e) {
        console.error("DB update error restoreMedicine:", e);
    }
};

// DELETE MEDICINE PERMANENTLY
const deleteMedicinePermanently = async (id: string) => {
    const med = archivedMedicines.find((m) => m.id === id);
    if (!med) return;

    const updatedArchived = archivedMedicines.filter((m) => m.id !== id);
    setArchivedMedicines(updatedArchived);
    saveToLocal("mswd_archived_medicines", updatedArchived);

    addToast(`Permanently deleted medicine ${med.name}.`, "error");

    addLog("MEDICINE_DELETED_PERMANENTLY", id, "medicines", { name:
med.name });

    try {
        const supabase = createClient();
        await supabase.from("medicines").delete().eq("id", id);
    } catch (e) {
        console.error("DB update error deleteMedicinePermanently:", e);
    }
};

// UPDATE MEDICINE REQUEST STATUS (With inventory deduction on dispatch)
const updateMedicineRequestStatus = async (

```

```

    id: string,
    status: MedicineRequestStatus,
    pharmacistNotes?: string
  ) => {
    const request = medicineRequests.find((r) => r.id === id);
    if (!request) return;

    // Handle inventory checks
    if ((status === "APPROVED" || status === "COMPLETED") &&
request.status === "PENDING") {
      const med = medicines.find((m) => m.id === request.medicine_id);
      if (med && med.available_quantity < request.quantity) {
        addToast(`Insufficient stock for ${med.name}. Stock is
${med.available_quantity}, requested ${request.quantity}.`, "error");
        return;
      }
    }

    const updatedRequests = medicineRequests.map((r) => {
      if (r.id === id) {
        const update: Partial<MedicineRequest> = { status };
        if (pharmacistNotes !== undefined) update.pharmacist_notes =
pharmacistNotes;
        if (status === "COMPLETED") {
          update.dispense_date = new Date().toISOString();
        }
        return { ...r, ...update } as MedicineRequest;
      }
      return r;
    });

    setMedicineRequests(updatedRequests);
    saveToLocal("mswd_med_requests", updatedRequests);

    // If request transitions to approved/completed from pending/rejected,
deduct stock
    if (
      (status === "APPROVED" || status === "COMPLETED") &&
      (request.status === "PENDING" || request.status === "REJECTED")
    ) {

```

```

    setMedicines((prev) => {
      const updated = prev.map((m) => {
        if (m.id === request.medicine_id) {
          return {
            ...m,
            available_quantity: Math.max(0, m.available_quantity -
request.quantity),
          };
        }
        return m;
      });
      saveToLocal("mswd_medicines", updated);
      return updated;
    });
  }

  // If request transitions to rejected from approved/completed, restore
stock
  if (
    status === "REJECTED" &&
    (request.status === "APPROVED" || request.status === "COMPLETED")
  ) {
    setMedicines((prev) => {
      const updated = prev.map((m) => {
        if (m.id === request.medicine_id) {
          return {
            ...m,
            available_quantity: m.available_quantity + request.quantity,
          };
        }
        return m;
      });
      saveToLocal("mswd_medicines", updated);
      return updated;
    });
  }

  const senior = seniors.find((s) => s.id === request.user_id);
  const med = medicines.find((m) => m.id === request.medicine_id);

```

```

    addToast(`Medicine request for ${senior?.full_name} marked as
    ${status}.`, "success");

    addLog(
      `MEDICINE_REQUEST_${status}`,
      id,
      "medicine_requests",
      {
        senior_name: senior?.full_name,
        medicine_name: med?.name,
        quantity: request.quantity,
        notes: pharmacistNotes,
      }
    );

    try {
      const supabase = createClient();
      await supabase
        .from("medicine_requests")
        .update({
          status,
          pharmacist_notes: pharmacistNotes,
          dispense_date: status === "COMPLETED" ? new Date().toISOString()
: null,
        })
        .eq("id", id);
    } catch (e) {
      console.error("DB update error updateMedicineRequestStatus:", e);
    }
  };

  // UPDATE ASSISTANCE REQUEST STATUS
  const updateAssistanceRequestStatus = async (
    id: string,
    status: AssistanceRequestStatus,
    notes?: string,
    assignedTo?: string | null
  ) => {
    const request = assistanceRequests.find((r) => r.id === id);
    if (!request) return;

```

```

const updatedRequests = assistanceRequests.map((r) => {
  if (r.id === id) {
    const update: Partial<AssistanceRequest> = { status };
    if (assignedTo !== undefined) update.assigned_to = assignedTo;
    return { ...r, ...update } as AssistanceRequest;
  }
  return r;
});

setAssistanceRequests(updatedRequests);
saveToLocal("mswd_ast_requests", updatedRequests);

const senior = seniors.find((s) => s.id === request.user_id);
addToast(`Assistance request (${request.category}) marked as
${status}.`, "success");

addLog(
  `ASSISTANCE_REQUEST_${status}`,
  id,
  "assistance_requests",
  {
    senior_name: senior?.full_name,
    category: request.category,
    assigned_to: assignedTo,
    admin_notes: notes,
  }
);

try {
  const supabase = createClient();
  await supabase
    .from("assistance_requests")
    .update({
      status,
      assigned_to: assignedTo,
    })
    .eq("id", id);
} catch (e) {
  console.error("DB update error updateAssistanceRequestStatus:", e);
}

```

```

    }
  };

  // REQUEST RESUBMISSION
  const requestResubmission = async (id: string, fields: string[], reason:
string) => {
    const updatedSeniors = seniors.map((s) => {
      if (s.id === id) {
        return {
          ...s,
          verification_status: "NEEDS_RESUBMISSION" as VerificationStatus,
          resubmit_fields: fields,
          id_document: {
            ...s.id_document,
            rejection_reason: reason,
          }
        };
      }
      return s;
    });
    setSeniors(updatedSeniors);
    saveToLocal("mswd_seniors", updatedSeniors);

    const senior = seniors.find((s) => s.id === id);
    addToast(`Requested updates from ${senior?.full_name ?? "Senior"}`,
"info");
    addLog(
      "RESUBMISSION_REQUESTED",
      id,
      "user_profiles",
      { name: senior?.full_name, fields, reason }
    );

    try {
      const supabase = createClient();
      await supabase.from("user_profiles").update({
        verification_status: "NEEDS_RESUBMISSION",
        resubmit_fields: fields
      }).eq("id", id);
      await supabase.from("id_documents").update({

```

```

        rejection_reason: reason
    }).eq("user_id", id);
} catch (e) {
    console.error("DB update error requestResubmission:", e);
}
};

// APPROVE RESUBMISSION
const approveResubmission = async (id: string) => {
    const senior = seniors.find((s) => s.id === id);
    if (!senior || !senior.pending_resubmission) return;

    const { resubmitted_data } = senior.pending_resubmission;

    const updatedSeniors = seniors.map((s) => {
        if (s.id === id) {
            const updatedAddress = { ...s.address };
            if (resubmitted_data.street !== undefined) updatedAddress.street =
resubmitted_data.street;
            if (resubmitted_data.barangay !== undefined)
updatedAddress.barangay = resubmitted_data.barangay;
            if (resubmitted_data.latitude !== undefined)
updatedAddress.latitude = resubmitted_data.latitude;
            if (resubmitted_data.longitude !== undefined)
updatedAddress.longitude = resubmitted_data.longitude;

            const updatedDoc = { ...s.id_document };
            if (resubmitted_data.id_front_url) updatedDoc.file_url =
resubmitted_data.id_front_url;

            return {
                ...s,
                full_name: resubmitted_data.full_name || s.full_name,
                verification_status: "APPROVED" as VerificationStatus,
                resubmit_fields: [],
                pending_resubmission: undefined,
                address: updatedAddress,
                id_document: {
                    ...updatedDoc,
                    verification_status: "APPROVED" as DocumentVerificationStatus,

```

```

        rejection_reason: undefined,
    }
};

}

return s;

});

setSeniors(updatedSeniors);
saveToLocal("mswd_seniors", updatedSeniors);

addToast(`Approved resubmitted details for
${resubmitted_data.full_name || senior.full_name}.`, "success");
addLog(
    "RESUBMISSION_APPROVED",
    id,
    "user_profiles",
    { name: resubmitted_data.full_name || senior.full_name }
);

try {
    const supabase = createClient();

    const profileUpdates: any = {
        verification_status: "APPROVED",
        resubmit_fields: []
    };

    if (resubmitted_data.full_name) profileUpdates.full_name =
resubmitted_data.full_name;
    if (resubmitted_data.pin) profileUpdates.login_pin =
resubmitted_data.pin;
    await supabase.from("user_profiles").update(profileUpdates).eq("id",
id);

    const addressUpdates: any = {};
    if (resubmitted_data.street !== undefined) addressUpdates.street =
resubmitted_data.street;
    if (resubmitted_data.barangay !== undefined) addressUpdates.barangay
= resubmitted_data.barangay;
    if (resubmitted_data.latitude !== undefined) addressUpdates.latitude
= resubmitted_data.latitude;

```

```

    if (resubmitted_data.longitude !== undefined)
addressUpdates.longitude = resubmitted_data.longitude;
    if (Object.keys(addressUpdates).length > 0) {
      await
supabase.from("user_addresses").update(addressUpdates).eq("user_id", id);
    }

    const docUpdates: any = { verification_status: "APPROVED",
rejection_reason: null };
    if (resubmitted_data.id_front_url) {
      await supabase.from("id_documents").update({ ...docUpdates,
file_url: resubmitted_data.id_front_url }).eq("user_id", id).eq("id_type",
"SENIOR_CITIZEN_ID_FRONT");
    } else {
      await
supabase.from("id_documents").update(docUpdates).eq("user_id", id);
    }

    if (resubmitted_data.id_back_url) {
      await supabase.from("id_documents").update({ file_url:
resubmitted_data.id_back_url, verification_status: "APPROVED",
rejection_reason: null }).eq("user_id", id).eq("id_type",
"SENIOR_CITIZEN_ID_BACK");
    }

    await supabase.from("resubmissions").update({ status: "APPROVED"
}).eq("id", senior.pending_resubmission.id);

    // Auto-generate Digital ID after approval
    generateDigitalId(id);
  } catch (e) {
    console.error("DB update error approveResubmission:", e);
  }
};

return (
  <AdminContext.Provider
    value={{
      seniors,
      medicines,

```

```

        medicineRequests,
        assistanceRequests,
        digitalIds,
        logs,
        toasts,
        addToast,
        removeToast,
        archivedSeniors,
        archivedMedicines,
        verifySenior,
        rejectSenior,
        flagSenior,
        updateSeniorNotes,
        deactivateSenior,
        restoreSenior,
        deleteSeniorPermanently,
        generateDigitalId,
        batchGenerateDigitalIds,
        suspendDigitalId,
        renewDigitalId,
        addMedicine,
        updateMedicine,
        deleteMedicine,
        restoreMedicine,
        deleteMedicinePermanently,
        updateMedicineRequestStatus,
        updateAssistanceRequestStatus,
        requestResubmission,
        approveResubmission,
    }}
    >
    {children}

    </AdminContext.Provider>
  );
}

export function useAdminStore() {
  const context = useContext(AdminContext);
  if (!context) {

```

```
    throw new Error("useAdminStore must be used within an AdminProvider");
  }
  return context;
}
```

"use server";

```
import { createClient } from "@lib/supabase/server";

export async function fetchMedicines() {
  const supabase = await createClient();

  const { data, error } = await supabase
    .from("medicines")
    .select("id, name, generic_name, description, dosage_strength, unit,
available_quantity")
    .eq("is_active", true)
    .order("name", { ascending: true });

  if (error) {
    console.error("Error fetching medicines:", error);
    return [];
  }

  return data || [];
}

export async function fetchUserRequests() {
  const supabase = await createClient();

  const {
    data: { user },
  } = await supabase.auth.getUser();

  if (!user) {
    return { medicineRequests: [], assistanceRequests: [] };
  }

  // Fetch medicine requests
```

```

const { data: medicineRequests, error: medError } = await supabase
  .from("medicine_requests")
  .select("id, medicine_id, quantity, reason, prescription_url, status,
pharmacist_notes, request_date, medicines(name, generic_name, unit)")
  .eq("user_id", user.id)
  .order("request_date", { ascending: false });

if (medError) {
  console.error("Error fetching medicine requests:", medError);
}

// Fetch assistance requests
const { data: assistanceRequests, error: assistError } = await supabase
  .from("assistance_requests")
  .select("id, category, description, urgency_level, status,
created_at")
  .eq("user_id", user.id)
  .order("created_at", { ascending: false });

if (assistError) {
  console.error("Error fetching assistance requests:", assistError);
}

// Format responses to match UI expectations
const formattedMedicines = (medicineRequests || []).map((req: any) => ({
  id: req.id,
  medicineName: req.medicines?.name || "Unknown Medicine",
  genericName: req.medicines?.generic_name || "",
  quantity: req.quantity,
  unit: req.medicines?.unit || "tablets",
  requestDate: new Date(req.request_date).toLocaleDateString("en-PH", {
    month: "long",
    day: "numeric",
    year: "numeric",
  }),
  status: req.status.toLowerCase() as "pending" | "approved" |
"completed" | "rejected",
  pharmacistNotes: req.pharmacist_notes || "",
  prescriptionUrl: req.prescription_url || "",
}));

```

```

    const formattedAssistance = (assistanceRequests || []).map((req: any) =>
    ({
      id: req.id,
      medicineName:
        req.category === "medical" ? "Medical Support" :
        req.category === "transport" ? "Transportation Support" :
        req.category === "healthcare" ? "Healthcare Support" :
        req.category === "social" ? "Social / Community Support" :
        req.category === "other" ? "Other Support" : "Food / Financial
Assistance",
      genericName: req.description,
      quantity: 1,
      unit: "request",
      requestDate: new Date(req.created_at).toLocaleDateString("en-PH", {
        month: "long",
        day: "numeric",
        year: "numeric",
      }),
      status: req.status.toLowerCase() as "pending" | "approved" |
"completed" | "rejected",
      pharmacistNotes: "",
    }));

    return {
      medicineRequests: formattedMedicines,
      assistanceRequests: formattedAssistance,
    };
  }

export async function submitMedicineRequest(formData: FormData) {
  const supabase = await createClient();

  const {
    data: { user },
  } = await supabase.auth.getUser();

  if (!user) {
    return { error: "Not authenticated" };
  }

```

```

const medicineId = formData.get("medicineId") as string;
const quantity = parseInt(formData.get("quantity") as string) || 1;
const notes = formData.get("notes") as string | null;
const prescriptionFile = formData.get("prescriptionFile") as File |
null;

if (!medicineId) {
  return { error: "Medicine ID is required" };
}

let prescriptionUrl = null;
if (prescriptionFile && prescriptionFile.size > 0) {
  const fileExt = prescriptionFile.name.split(".").pop();
  const filePath = `${user.id}/prescription_${Date.now()}.${fileExt}`;

  const { error: uploadError } = await supabase.storage
    .from("id-documents")
    .upload(filePath, prescriptionFile, {
      contentType: prescriptionFile.type,
      upsert: false,
    });

  if (uploadError) {
    console.error("Upload error (prescription):", uploadError);
    return { error: "Failed to upload prescription image." };
  }

  const {
    data: { publicUrl },
  } = supabase.storage.from("id-documents").getPublicUrl(filePath);

  prescriptionUrl = publicUrl;
}

const { error } = await supabase
  .from("medicine_requests")
  .insert({
    user_id: user.id,
    medicine_id: medicineId,

```

```

        quantity,
        reason: notes || null,
        prescription_url: prescriptionUrl,
        status: "PENDING",
    });

    if (error) {
        console.error("Error inserting medicine request:", error);
        return { error: error.message };
    }

    return { success: true };
}

export async function submitAssistanceRequest(category: string,
description: string) {
    const supabase = await createClient();

    const {
        data: { user },
    } = await supabase.auth.getUser();

    if (!user) {
        return { error: "Not authenticated" };
    }

    // Get user address if exists
    const { data: address } = await supabase
        .from("user_addresses")
        .select("id")
        .eq("user_id", user.id)
        .limit(1)
        .maybeSingle();

    const { error } = await supabase
        .from("assistance_requests")
        .insert({
            user_id: user.id,
            category,
            description,

```

```

        status: "PENDING",
        address_id: address?.id || null,
    });

    if (error) {
        console.error("Error inserting assistance request:", error);
        return { error: error.message };
    }

    return { success: true };
}

```

"use client";

```

import { useState, useEffect, useRef } from "react";
import { useSearchParams, useRouter } from "next/navigation";
import { Button } from "@/components/ui/button";
import { Input } from "@/components/ui/input";
import { Label } from "@/components/ui/label";
import {
    Select,
    SelectContent,
    SelectItem,
    SelectTrigger,
    SelectValue,
} from "@/components/ui/select";
import { Textarea } from "@/components/ui/textarea";
import { Tabs, TabsContent, TabsList, TabsTrigger } from
"@/components/ui/tabs";
import {
    Minus,
    Plus,
    Send,
    Heart,
    HandHeart,
    Loader2,
    Handshake,
    AlertCircle,
    Upload,

```

```

X,
CheckCircle2,
ClipboardList,
Inbox,
} from "lucide-react";
import { MedicineRequestCard } from
"@/components/user/medicine-request-card";
import { SectionHeader } from "@/components/user/section-header";
import { useLanguage } from "@/context/LanguageContext";
import { toast } from "sonner";
import {
  fetchMedicines,
  fetchUserRequests,
  submitMedicineRequest,
  submitAssistanceRequest,
} from "@/app/actions/requests";
import { clientCache } from "@/lib/client-cache";

export function RequestsSectionsContent() {
  const { t, language } = useLanguage();
  const searchParams = useSearchParams();
  const router = useRouter();

  // Get active tab from URL search parameters, default to "medicine"
  const tabParam = searchParams.get("tab");
  const [activeTab, setActiveTab] = useState<string>("medicine");

  useEffect(() => {
    if (tabParam === "medicine" || tabParam === "assistance") {
      // eslint-disable-next-line react-hooks/set-state-in-effect
      setActiveTab(tabParam);
    }
  }, [tabParam]);

  const handleTabChange = (val: string) => {
    setActiveTab(val);
    router.replace(`/requests?tab=${val}`);
  };

  const [quantity, setQuantity] = useState(1);

```

```

const [medicine, setMedicine] = useState<string>("");
const [assistanceType, setAssistanceType] = useState<string>("");
const [medicineNotes, setMedicineNotes] = useState<string>("");
const [assistanceReason, setAssistanceReason] = useState<string>("");
const [prescriptionFile, setPrescriptionFile] = useState<File |
null>(null);
const [prescriptionPreview, setPrescriptionPreview] = useState<string |
null>(
  null,
);
const fileInputRef = useRef<HTMLInputElement>(null);

const [availableMedicines, setAvailableMedicines] = useState<any[]>([]);
const [medicineRequests, setMedicineRequests] = useState<any[]>([]);
const [assistanceRequests, setAssistanceRequests] = useState<any[]>([]);
const [loading, setLoading] = useState(true);
const [submittingMed, setSubmittingMed] = useState(false);
const [submittingAssist, setSubmittingAssist] = useState(false);
const [showRequestsSheet, setShowRequestsSheet] = useState(false);
const [isClosingSheet, setIsClosingSheet] = useState(false);

const handleFileChange = (e: React.ChangeEvent<HTMLInputElement>) => {
  const file = e.target.files?.[0];
  if (file) {
    if (file.size > 5 * 1024 * 1024) {
      toast.error(
        language === "tl"
          ? "Masyadong malaki ang file (Max 5MB)"
          : "File size too large (Max 5MB)",
      );
      return;
    }
    setPrescriptionFile(file);
    const reader = new FileReader();
    reader.onloadend = () => {
      setPrescriptionPreview(reader.result as string);
    };
    reader.readAsDataURL(file);
  }
};

```

```

const removeFile = () => {
  setPrescriptionFile(null);
  setPrescriptionPreview(null);
  if (fileInputRef.current) {
    fileInputRef.current.value = "";
  }
};

const closeRequestsSheet = () => {
  setIsClosingSheet(true);
  setTimeout(() => {
    setShowRequestsSheet(false);
    setIsClosingSheet(false);
  }, 250);
};

useEffect(() => {
  if (showRequestsSheet) {
    document.body.style.overflow = "hidden";
  } else {
    document.body.style.overflow = "unset";
  }
  return () => {
    document.body.style.overflow = "unset";
  };
}, [showRequestsSheet]);

useEffect(() => {
  async function loadData() {
    // 1. Read from client cache first to prevent redundant loader
states
    const cachedMeds = clientCache.getMedicines();
    const cachedReqs = clientCache.getUserRequests();

    if (cachedMeds) {
      setAvailableMedicines(cachedMeds);
    }

    if (cachedReqs) {
      setMedicineRequests(cachedReqs.medicineRequests);
    }
  }
}, []);

```

```

        setAssistanceRequests(cachedReqs.assistanceRequests);
        setLoading(false);
    }

    try {
        const [meds, reqs] = await Promise.all([
            cachedMeds ? Promise.resolve(cachedMeds) : fetchMedicines(),
            fetchUserRequests(),
        ]);

        if (meds) {
            clientCache.setMedicines(meds);
            setAvailableMedicines(meds);
        }

        if (reqs) {
            clientCache.setUserRequests(reqs);
            setMedicineRequests(reqs.medicineRequests);
            setAssistanceRequests(reqs.assistanceRequests);
        }
    } catch (err) {
        console.error("Error loading data:", err);
    } finally {
        setLoading(false);
    }
}

loadData();
}, []);

const countWords = (text: string) => {
    if (!text.trim()) return 0;
    return text.trim().split(/\s+/).length;
};

const medicineWordCount = countWords(medicineNotes);
const assistanceWordCount = countWords(assistanceReason);

const selectedMedicine = availableMedicines.find((m) => m.id ===
medicine);
const maxQuantity = selectedMedicine?.available_quantity ?? 9999;

```

```

const handleMedicineSubmit = async () => {
  if (!medicine) return;

  if (quantity > maxQuantity) {
    const medicineUnit = selectedMedicine?.unit || "units";
    toast.error(
      language === "tl"
        ? `Mayroon lang ${maxQuantity} ${medicineUnit} available na stocks.`
        : `Only ${maxQuantity} ${medicineUnit} available in stock.`,
    );
    return;
  }

  setSubmittingMed(true);
  try {
    const formData = new FormData();
    formData.append("medicineId", medicine);
    formData.append("quantity", quantity.toString());
    formData.append("notes", medicineNotes);
    if (prescriptionFile) {
      formData.append("prescriptionFile", prescriptionFile);
    }

    const res = await submitMedicineRequest(formData);
    if (res.error) {
      toast.error(res.error);
    } else {
      setMedicine("");
      setQuantity(1);
      setMedicineNotes("");
      removeFile();
      const reqs = await fetchUserRequests();
      if (reqs) {
        clientCache.setUserRequests(reqs);
        setMedicineRequests(reqs.medicineRequests);
      }
      toast.success(
        language === "tl"
          ? "Matagumpay na naipadala ang kahilingan!"

```

```

        : "Request submitted successfully!",
    );
}
} catch (err) {
    console.error(err);
    toast.error(
        language === "tl"
        ? "May problema sa pagpadala ng kahilingan."
        : "Failed to submit request.",
    );
} finally {
    setSubmittingMed(false);
}
};

const handleAssistanceSubmit = async () => {
    if (!assistanceType || !assistanceReason.trim()) return;
    setSubmittingAssist(true);
    try {
        const res = await submitAssistanceRequest(
            assistanceType,
            assistanceReason,
        );
        if (res.error) {
            toast.error(res.error);
        } else {
            setAssistanceType("");
            setAssistanceReason("");
            const reqs = await fetchUserRequests();
            if (reqs) {
                clientCache.setUserRequests(reqs);
                setAssistanceRequests(reqs.assistanceRequests);
            }
            toast.success(
                language === "tl"
                ? "Matagumpay na naipadala ang kahilingan!"
                : "Request submitted successfully!",
            );
        }
    } catch (err) {

```

```

        console.error(err);
        toast.error(
            language === "tl"
                ? "May problema sa pagpadala ng kahilingan."
                : "Failed to submit request.",
        );
    } finally {
        setSubmittingAssist(false);
    }
};

return (
    <Tabs
        value={activeTab}
        onChange={handleTabChange}
        className="w-full flex flex-col gap-6"
    >
        <TabsList className="grid w-full grid-cols-2 h-11 bg-muted p-1
rounded-lg">
            <TabsTrigger
                value="medicine"
                className="gap-2 text-sm font-medium rounded-md"
            >
                <Heart className="h-4 w-4" />
                {t.medicineLabel}
            </TabsTrigger>
            <TabsTrigger
                value="assistance"
                className="gap-2 text-sm font-medium rounded-md"
            >
                <Handshake className="h-4 w-4" />
                {t.assistance}
            </TabsTrigger>
        </TabsList>

        {/* Medicine Tab Content */}
        <TabsContent
            value="medicine"
            className="flex flex-col gap-8 mt-0 pb-24
focus-visible:outline-none"

```

```

>
  {/** Section 1: Request Medicine */}
  <section className="flex flex-col gap-3">
    <SectionHeader title={t.requestMedicine} />
    <div className="rounded border border-border bg-card p-5 flex
flex-col gap-5">
      {/** Medicine Selector */}
      <div className="flex flex-col gap-2">
        <Label htmlFor="medicine" className="text-base font-medium">
          {t.medicineLabel}
        </Label>
        <Select value={medicine} onChange={setMedicine}>
          <SelectTrigger id="medicine" className="w-full h-10">
            <SelectValue placeholder={t.selectMedicine} />
          </SelectTrigger>
          <SelectContent>
            {availableMedicines.map((med) => (
              <SelectItem key={med.id} value={med.id}>
                {med.name} {" "}
                {med.dosage_strength ? `(${med.dosage_strength})` :
""}

              </SelectItem>
            ))}
          </SelectContent>
        </Select>
      </div>

      {/** Quantity Stepper */}
      <div className="flex flex-col gap-2">
        <Label className="text-base
font-medium">{t.quantity}</Label>
        <div className="flex items-center gap-3">
          <Button
            type="button"
            variant="outline"
            size="icon"
            className="h-10 w-10 shrink-0 rounded-md"
            onClick={() => setQuantity(Math.max(1, quantity - 1))}
            disabled={quantity <= 1}
          >

```

```

        <Minus className="h-4 w-4" />
      </Button>
      <Input
        type="number"
        value={quantity}
        onChange={(e) =>
          setQuantity(Math.max(1, parseInt(e.target.value) ||
1))
        }
        className="h-10 text-center text-lg font-semibold
rounded-md"
        min={1}
      />
      <Button
        type="button"
        variant="outline"
        size="icon"
        className="h-10 w-10 shrink-0 rounded-md"
        onClick={() => setQuantity(quantity + 1)}
      >
        <Plus className="h-4 w-4" />
      </Button>
    </div>
  </div>

  { /* Notes */ }
  <div className="flex flex-col gap-2">
    <div className="flex justify-between items-center">
      <Label htmlFor="notes" className="text-base font-medium">
        {t.notes}{" "}
        <span className="text-muted-foreground font-normal">
          {t.optional}
        </span>
      </Label>
      <span
        className={`text-sm ${
          medicineWordCount > 300
            ? "text-destructive font-medium"
            : "text-muted-foreground"
        }`}
      >

```

```

    >
    {medicineWordCount} / 300{" "}
    {language === "tl" ? "salita" : "words"}
  </span>
</div>
<Textarea
  id="notes"
  value={medicineNotes}
  onChange={(e) => setMedicineNotes(e.target.value)}
  placeholder={t.notesPlaceholder}
  className={`w-full min-h-20 resize-none ${
    medicineWordCount > 300
      ? "border-destructive
focus-visible:ring-destructive/20"
      : ""
    }`}
/>
{medicineWordCount > 300 && (
  <span className="text-xs text-destructive font-medium">
    {language === "tl"
      ? "Masyadong mahaba (300 salita max)"
      : "Too long (300 words max)"}
    </span>
  )}
</div>

{/* Prescription Upload */}
<div className="flex flex-col gap-2">
  <Label className="text-base font-medium flex items-center
justify-between">
    <span>
      {language === "tl"
        ? "Reseta ng Doktor"
        : "Doctor's Prescription"}{" "}
      <span className="text-muted-foreground font-normal
text-sm">
        ({t.optional})
      </span>
    </span>
  </Label>

```

```

        {prescriptionPreview ? (
          <div className="relative rounded-xl overflow-hidden border
border-border">
            <img
              src={prescriptionPreview}
              alt="Prescription Preview"
              className="w-full h-40 object-cover"
            />
            <button
              type="button"
              onClick={removeFile}
              className="absolute top-2 right-2 p-1.5 rounded-full
bg-background/80 backdrop-blur-sm hover:bg-background transition-colors
cursor-pointer"
            >
              <X className="w-4 h-4 text-foreground" />
            </button>
            <div className="absolute bottom-2 left-2 flex
items-center gap-1.5 px-2.5 py-1 rounded-full bg-background/80
backdrop-blur-sm">
              <CheckCircle2 className="w-3.5 h-3.5 text-emerald-600
shrink-0" />
              <span className="text-xs font-medium truncate
max-w-[200px]">
                {prescriptionFile?.name}
              </span>
            </div>
          </div>
        ) : (
          <button
            type="button"
            onClick={() => fileInputRef.current?.click()}
            className="flex flex-col items-center justify-center
gap-2 h-24 rounded-xl border border-dashed border-border bg-muted/40
hover:bg-muted/70 transition-colors cursor-pointer"
          >
            <div className="flex items-center justify-center w-8 h-8
rounded-full bg-background border border-border">
              <Upload className="w-4 h-4 text-muted-foreground" />
            </div>
          </button>
        )
      }
    </div>
  )
}

```

```

        <div className="text-center">
          <p className="text-xs font-medium text-foreground">
            {language === "tl"
              ? "I-upload ang Reseta"
              : "Upload Prescription"}
          </p>
          <p className="text-[10px] text-muted-foreground">
            JPG | PNG | WEBP (Max 5MB)
          </p>
        </div>
      </button>
    )}
    <input
      ref={fileInputRef}
      type="file"
      accept="image/jpeg,image/png,image/webp"
      className="hidden"
      onChange={handleFileChange}
    />
  </div>

  { /* Submit */ }
  <Button
    onClick={handleMedicineSubmit}
    disabled={
      medicineWordCount > 300 ||
      !medicine ||
      submittingMed ||
      quantity > maxQuantity
    }
    className="h-10 w-full text-base font-medium gap-2
rounded-md"
  >
    {submittingMed ? (
      <Loader2 className="h-4 w-4 animate-spin" />
    ) : (
      <Send className="h-4 w-4" />
    )}
    {t.submitRequest}
  </Button>

```

```

        </div>
    </section>
</TabsContent>

{/* Assistance Tab Content */}
<TabsContent
    value="assistance"
    className="flex flex-col gap-8 mt-0 pb-24
focus-visible:outline-none"
>
    {/* Section 1: Request Assistance */}
    <section className="flex flex-col gap-3">
        <SectionHeader title={t.requestAssistance} />
        <div className="rounded border border-border bg-card p-5 flex
flex-col gap-5">
            {/* Assistance Type Selector */}
            <div className="flex flex-col gap-2">
                <Label htmlFor="assistanceType" className="text-base
font-medium">
                    {t.assistanceType}
                </Label>
                <Select value={assistanceType}
onValueChange={setAssistanceType}>
                    <SelectTrigger id="assistanceType" className="w-full
h-10">
                        <SelectValue placeholder={t.selectAssistanceType} />
                    </SelectTrigger>
                    <SelectContent>
                        <SelectItem value="medical">{t.medicalHelp}</SelectItem>
                        <SelectItem value="non-medical">
                            {t.nonMedicalHelp}
                        </SelectItem>
                        <SelectItem
value="transport">{t.transportHelp}</SelectItem>
                        <SelectItem
value="healthcare">{t.healthcareHelp}</SelectItem>
                        <SelectItem value="social">{t.socialHelp}</SelectItem>
                        <SelectItem value="other">{t.otherHelp}</SelectItem>
                    </SelectContent>
                </Select>
            </div>
        </div>
    </section>

```

```

</div>

{/* Reason/Description */}
<div className="flex flex-col gap-2">
  <div className="flex justify-between items-center">
    <Label htmlFor="reason" className="text-base font-medium">
      {t.assistanceReason}
    </Label>
    <span
      className={`text-sm ${
        assistanceWordCount > 500
          ? "text-destructive font-medium"
          : "text-muted-foreground"
        }`}
    >
      {assistanceWordCount} / 500{" "}
      {language === "tl" ? "salita" : "words"}
    </span>
  </div>
  <Textarea
    id="reason"
    value={assistanceReason}
    onChange={(e) => setAssistanceReason(e.target.value)}
    placeholder={t.assistanceReasonPlaceholder}
    className={`w-full min-h-24 resize-none ${
      assistanceWordCount > 500
        ? "border-destructive
focus-visible:ring-destructive/20"
        : ""
      }`}
  />
  {assistanceWordCount > 500 && (
    <span className="text-sm text-destructive font-medium">
      {language === "tl"
        ? "Masyadong mahaba (500 salita max)"
        : "Too long (500 words max)"}
    </span>
  )}
</div>

```

```

        { /* Submit */ }
        <Button
            onClick={handleAssistanceSubmit}
            disabled={
                assistanceWordCount > 500 ||
                !assistanceType ||
                !assistanceReason.trim() ||
                submittingAssist
            }
            className="h-10 w-full text-base font-medium gap-2
rounded-md"
        >
            {submittingAssist ? (
                <Loader2 className="h-4 w-4 animate-spin" />
            ) : (
                <Send className="h-4 w-4" />
            )}
            {t.submitRequest}
        </Button>
    </div>
</section>

{ /* Section 2: My Assistance Requests */ }
<section className="flex flex-col gap-3">
    <SectionHeader
        title={t.myRequests}
        actionLabel={t.seeAll}
        actionHref="/requests/history"
    />
    <div className="flex flex-col gap-2.5">
        {assistanceRequests.length === 0 ? (
            <div className="rounded-xl border bg-card/30 p-6 text-center
text-sm text-muted-foreground">
                {language === "tl"
                    ? "Walang mga kahilingan sa tulong"
                    : "No assistance requests found"}
            </div>
        ) : (
            assistanceRequests.map((request) => (
                <MedicineRequestCard key={request.id} request={request} />
            ))
        )}
    </div>
</section>

```

```

        ))
      })
    </div>
  </section>
</TabsContent>

  {/* Floating Button */}
  <div className="fixed bottom-22 max-w-sm w-full left-1/2
-translate-x-1/2 px-5 flex justify-end pointer-events-none z-40">
    <Button
      onClick={() => setShowRequestsSheet(true)}
      className="relative w-12 h-12 rounded-full shadow-lg flex
items-center justify-center pointer-events-auto bg-primary
hover:bg-primary/90 text-primary-foreground transition-transform
hover:scale-105 active:scale-95 cursor-pointer"
    >
      <Inbox size={24} />
      {(activeTab === "medicine"
        ? medicineRequests.length
        : assistanceRequests.length) > 0 && (
        <span className="absolute -top-1 -right-1 w-5 h-5 rounded-full
bg-red-500 text-white text-[10px] font-semibold flex items-center
justify-center">
          {activeTab === "medicine"
            ? medicineRequests.length
            : assistanceRequests.length}
        </span>
      )}
    </Button>
  </div>

  {/* Bottom Sheet */}
  {showRequestsSheet && (
    <div className="fixed inset-0 z-[100] flex items-end
justify-center">
      <div
        className="absolute inset-0 bg-black/50 animate-in fade-in
duration-300"
        onClick={closeRequestsSheet}
      />

```

```

<div
  className={`absolute bottom-0 bg-card rounded-t-2xl
max-h-[80vh] w-full max-w-sm flex flex-col border-t border-border ${
    isClosingSheet
      ? "animate-out slide-out-to-bottom duration-300"
      : "animate-in slide-in-from-bottom duration-300"
  }}
>
  <div className="flex items-center justify-between px-6 py-3
border-b shrink-0">
    <p className="font-semibold text-base text-foreground">
      {activeTab === "medicine"
        ? language === "tl"
          ? "Mga Kahilingan sa Gamot"
          : "Medicine Requests"
        : language === "tl"
          ? "Mga Kahilingan sa Tulong"
          : "Assistance Requests"}
    </p>
    <Button
      variant="ghost"
      size="icon"
      onClick={closeRequestsSheet}
      className="h-8 w-8 rounded-full hover:bg-muted"
    >
      <X size={16} />
    </Button>
  </div>

  <div className="px-6 py-4 overflow-y-auto flex-1 flex flex-col
gap-3">
    {activeTab === "medicine" ? (
      medicineRequests.length === 0 ? (
        <div className="py-8 text-center text-sm
text-muted-foreground">
          {language === "tl"
            ? "Walang mga kahilingan sa gamot"
            : "No medicine requests found"}
          </div>
        ) : (

```

```

        medicineRequests
        .slice(0, 3)
        .map((request) => (
            <MedicineRequestCard key={request.id}
request={request} />
        ))
    )
    ) : assistanceRequests.length === 0 ? (
        <div className="py-8 text-center text-sm
text-muted-foreground">
            {language === "tl"
                ? "Walang mga kahilingan sa tulong"
                : "No assistance requests found"}
        </div>
    ) : (
        assistanceRequests
        .slice(0, 3)
        .map((request) => (
            <MedicineRequestCard key={request.id}
request={request} />
        ))
    )}
</div>

<div className="px-6 py-4 border-t bg-muted/10 flex
items-center justify-between shrink-0">
    <span className="text-xs text-muted-foreground">
        {activeTab === "medicine"
            ? `${medicineRequests.length} total`
            : `${assistanceRequests.length} total`}
    </span>
    <a
        href="/requests/history"
        className="text-sm font-semibold text-primary
hover:underline"
        onClick={() => {
            document.body.style.overflow = "unset";
            setShowRequestsSheet(false);
        }}
    >

```

```

        {t.seeAll}
      </a>
    </div>
  </div>
</div>
  )}
</Tabs>
);
}

import { Suspense } from "react";

export function RequestsSections() {
  return (
    <Suspense
      fallback={
        <div className="text-center py-6 text-sm text-muted-foreground">
          Loading
        </div>
      }
    >
      <RequestsSectionsContent />
    </Suspense>
  );
}

```